Computational Geometry Algorithms & Applications

Steven Merino

December 31, 1979

Abstract

This report presents implementations of three fundamental computational geometry algorithms: Graham Scan for convex hull computation, Bentley-Ottmann for line segment intersection detection, and Welzl's algorithm for minimum enclosing circle. Each algorithm is explained intuitively with emphasis on the key insights that enable efficient computation. The implementations are developed in Zig with SDL3 visualization. As a practical application, we apply these algorithms to determine optimal antenna placement for covering the 34 parishes of Quito, Ecuador, using real geographic data from INEC Ecuador.

1 Introduction

Computational geometry provides algorithmic solutions to geometric problems that arise in computer graphics, robotics, geographic information systems, and numerous other domains. This work implements three classical algorithms, each demonstrating a different algorithmic paradigm:

- **Graham Scan**: A sorting-based approach achieving $O(n \log n)$ convex hull computation.
- **Bentley-Ottmann**: A sweep line algorithm detecting k intersections among n segments in $O((n + k) \log n)$ time.
- **Welzl**: A randomized incremental algorithm computing the minimum enclosing circle in expected $O(n)$ time.

2 Convex Hull: Graham Scan

2.1 Problem Definition

Given a set P of n points in the plane, the convex hull is the smallest convex polygon containing all points. Equivalently, it is the shape formed by stretching a rubber band around all points.

2.2 Algorithm Intuition

Graham Scan exploits a key observation: if we sort points by polar angle around an extreme point (the lowest, rightmost point), we can construct the hull by traversing points in order and maintaining only those that form left turns.

The algorithm proceeds in three phases:

1. **Find pivot**: Select the point with minimum y-coordinate (rightmost if tied). This point is guaranteed to be on the hull.
2. **Sort by polar angle**: Order remaining points by their angle relative to the pivot. Points at the same angle are ordered by distance.

3. **Stack-based scan**: Process points in sorted order, maintaining a stack of hull vertices. For each new point, pop vertices from the stack while they would create a right turn (or collinear segment), then push the new point.

2.3 The Turn Test

The core operation is determining turn direction using the cross product. For three consecutive points p_1, p_2, p_3, the cross product of vectors $\overrightarrow{p_1p_2}$ and $\overrightarrow{p_1p_3}$ indicates:

$$\text{cross}(p_1, p_2, p_3) = (p_2.x - p_1.x)(p_3.y - p_1.y) - (p_2.y - p_1.y)(p_3.x - p_1.x) \tag{1}$$

- Positive: counter-clockwise turn (left turn)
- Negative: clockwise turn (right turn)
- Zero: collinear points

2.4 Implementation

The cross product computation uses 64-bit integers to prevent overflow when input coordinates are 32-bit:

```
fn crossProduct(o: Point, a: Point, b: Point) i64 {
    const oa_x: i64 = @as(i64, a.x) - @as(i64, o.x);
    const oa_y: i64 = @as(i64, a.y) - @as(i64, o.y);
    const ob_x: i64 = @as(i64, b.x) - @as(i64, o.x);
    const ob_y: i64 = @as(i64, b.y) - @as(i64, o.y);
    return oa_x * ob_y - oa_y * ob_x;
}
```

Listing 1: Cross product with overflow prevention

2.5 Complexity Analysis

- Finding the pivot: $O(n)$
- Sorting by polar angle: $O(n \log n)$
- Stack-based scan: $O(n)$ amortized (each point is pushed and popped at most once)

Total complexity: $O(n \log n)$, which is optimal for comparison-based convex hull algorithms.

3 Segment Intersection: Bentley-Ottmann

3.1 Problem Definition

Given n line segments in the plane, find all k intersection points. A naive approach testing all pairs requires $O(n^2)$ time regardless of the number of intersections. Bentley-Ottmann achieves output-sensitive complexity.

3.2 Algorithm Intuition

The key insight is that segments can only intersect if they are "close" to each other. The sweep line paradigm exploits this by maintaining only segments that overlap at the current x-coordinate.

Imagine a vertical line sweeping left-to-right across the plane. At any position, only a subset of segments intersects the sweep line. These *active segments* are ordered by their y-coordinate at the current x-position. Two segments can only intersect if they become adjacent in this ordering.

3.3 Event-Driven Processing

The algorithm processes three types of events:

1. **Segment start**: A segment's left endpoint is encountered. Insert the segment into the active set and check for intersections with its neighbors.
2. **Segment end**: A segment's right endpoint is encountered. Before removing it, check if its former neighbors now intersect.
3. **Intersection**: Two segments cross. Swap their positions in the active set (their relative y-order reverses) and check for new intersections with their new neighbors.

3.4 Data Structures

- **Event queue**: A priority queue ordered by x-coordinate (then y, then event type). Implemented using `std.PriorityQueue`.
- **Sweep line status**: An ordered set of active segments sorted by y at current x. Implemented using a sorted `ArrayList` for simplicity.

3.5 Segment Intersection Test

Two segments s_1 from (x_1, y_1) to (x_2, y_2) and s_2 from (x_3, y_3) to (x_4, y_4) intersect at parameter values t and u:

$$t = \frac{(x_1 - x_3)(y_3 - y_4) - (y_1 - y_3)(x_3 - x_4)}{(x_1 - x_2)(y_3 - y_4) - (y_1 - y_2)(x_3 - x_4)} \tag{2}$$

The segments intersect if and only if $t, u \in [0, 1]$.

3.6 Complexity Analysis

- Each segment generates 2 events (start and end)
- Each intersection generates 1 event
- Total events: $O(n + k)$
- Each event involves $O(\log n)$ operations on the data structures

Total complexity: $O((n + k) \log n)$, which is superior to $O(n^2)$ when $k = o(n^2/\log n)$.

4 Minimum Enclosing Circle: Welzl's Algorithm

4.1 Problem Definition

Given n points in the plane, find the smallest circle (by radius) that contains all points. This circle is unique and is determined by at most three points on its boundary.

4.2 Algorithm Intuition

Welzl's algorithm is based on a elegant recursive observation: if we remove an arbitrary point p from the set and compute the minimum enclosing circle for the remaining points, then either:

1. Point p lies inside this circle, so it is also the answer for the full set.
2. Point p lies outside, so p must be on the boundary of the minimum enclosing circle for the full set.

This leads to a recursive algorithm that maintains a set of "boundary points" known to lie on the final circle.

4.3 The Algorithm

Algorithm 1 Welzl's Algorithm

```
function WELZL(P, R)
    if P = ∅ or |R| = 3 then
        return TRIVIALCIRCLE(R)
    end if
    Choose random p ∈ P
    D ← WELZL(P \ {p}, R)
    if p ∈ D then
        return D
    end if
    return WELZL(P \ {p}, R ∪ {p})
end function
```

The TRIVIALCIRCLE function computes the unique circle through 0, 1, 2, or 3 points:

- 0 points: degenerate circle with radius 0
- 1 point: circle centered at the point with radius 0
- 2 points: circle with the points as diameter endpoints
- 3 points: circumcircle (or diameter of farthest pair if collinear)

4.4 Circumcircle Computation

For three non-collinear points (a_x, a_y), (b_x, b_y), (c_x, c_y), the circumcenter (u_x, u_y) is:

$$d = 2(a_x(b_y - c_y) + b_x(c_y - a_y) + c_x(a_y - b_y)) \tag{3}$$

$$u_x = \frac{(a_x^2 + a_y^2)(b_y - c_y) + (b_x^2 + b_y^2)(c_y - a_y) + (c_x^2 + c_y^2)(a_y - b_y)}{d} \tag{4}$$

This requires floating-point arithmetic, hence the implementation uses `f64` for circle computations:

```zig
pub const Circle = struct {
    center: PointF64,
    radius: f64,

    pub fn containsPoint(self: Circle, p: PointF64) bool {
        const epsilon = 1e-9;
        return self.center.distanceSquaredTo(p) <=
               (self.radius + epsilon) * (self.radius + epsilon);
    }
};
```

Listing 2: Circumcircle computation

4.5 Complexity Analysis

The expected running time is $O(n)$. The key insight is that the probability of a point being on the boundary decreases as more points are processed. Specifically, if we process points in random order, the expected number of times we recurse with an enlarged boundary set is constant.

The worst-case complexity is $O(n!)$ but occurs with negligible probability when points are randomly shuffled.

5 Application: Antenna Network in Quito

5.1 Problem Statement

A telecommunications company wants to cover n neighborhoods (parishes) of Quito with a single antenna that minimizes the coverage radius. This is a direct application of the minimum enclosing circle problem to real geographic data.

The objectives are:

1. Compute the minimum enclosing circle for parish centroids
2. Visualize the points, circle, and convex hull overlaid
3. Compare the "wasted area" between circle and convex hull
4. Report the optimal antenna location in geographic coordinates

5.2 Data Acquisition

Real parish boundary data was obtained from INEC Ecuador (Instituto Nacional de Estadística y Censos) via their public ArcGIS FeatureServer:

```bash
curl -s "https://services7.arcgis.com/iFGeGXTAJXnjq0YN/\
ArcGIS/rest/services/Parroquias_del_Ecuador/FeatureServer/0/\
query?where=DPA_DESCAN='QUITO'&outFields=DPA_PARROQ,DPA_DESPAR\
&returnGeometry=true&outSR=4326&f=geojson" \
> data/quito_parroquias.geojson
```

Listing 3: Fetching GeoJSON data from INEC Ecuador

This query returns 34 parishes in the Quito canton, each with MultiPolygon geometry defining its boundaries and properties including the parish name (`DPA_DESPAR`) and code (`DPA_PARROQ`).

5.3 Data Preprocessing

The GeoJSON contains complex MultiPolygon geometries. For the antenna placement problem, we need a single representative point (centroid) for each parish. A Python script extracts these centroids:

```python
import json

def multipolygon_centroid(coords):
    """Use largest polygon's centroid (by area)"""
    max_area = 0
    best_centroid = (0, 0)
    for polygon in coords:
        ring = polygon[0]  # exterior ring
        n = len(ring)
        # Shoelace formula for area
        area = abs(sum(ring[i][0] * ring[(i+1)%n][1] -
                       ring[(i+1)%n][0] * ring[i][1]
                       for i in range(n))) / 2
        if area > max_area:
            max_area = area
            # Centroid as average of vertices
            best_centroid = (sum(p[0] for p in ring) / n,
                             sum(p[1] for p in ring) / n)
    return best_centroid

data = json.load(open("data/quito_parroquias.geojson"))
for feature in data["features"]:
    name = feature["properties"]["DPA_DESPAR"]
    lon, lat = multipolygon_centroid(
        feature["geometry"]["coordinates"])
    print(f"{name},{lat:.6f},{lon:.6f}")
```

Listing 4: Centroid extraction from GeoJSON

The output is a CSV file with 34 parish centroids, ranging from PACTO (lat 0.161ř, lon -78.853ř) in the northwest to PINTAG (lat -0.365ř, lon -78.358ř) in the southeast.

5.4 Coordinate Transformation

Geographic coordinates (latitude/longitude) must be transformed for geometric computations. The implementation uses an equirectangular projection centered on Quito:

```rust
const EARTH_RADIUS_KM: f64 = 6371.0;
const QUITO_REF_LAT: f64 = -0.18;  // degrees
const QUITO_REF_LON: f64 = -78.47;

pub fn latLonToKm(lat: f64, lon: f64) struct { x: f64, y: f64 } {
    const lat_rad = lat * std.math.pi / 180.0;
    const lon_rad = lon * std.math.pi / 180.0;
    const ref_lat_rad = QUITO_REF_LAT * std.math.pi / 180.0;
    const ref_lon_rad = QUITO_REF_LON * std.math.pi / 180.0;

    // Equirectangular projection
    const x = EARTH_RADIUS_KM * (lon_rad - ref_lon_rad)
              * @cos(ref_lat_rad);
    const y = EARTH_RADIUS_KM * (lat_rad - ref_lat_rad);
    return .{ .x = x, .y = y };
}
```

Listing 5: Geographic to local coordinate conversion

This projection preserves distances reasonably well for the scale of Quito (approximately 70 km Œ 60 km).

5.5 Implementation Pipeline

The antenna placement runner combines the previously implemented algorithms:

1. **Load and parse**: Read embedded CSV data, extract parish centroids
2. **Transform coordinates**: Convert lat/lon to local km, then scale to [0,100] for the renderer
3. **Compute minimum enclosing circle**: Apply Welzl's algorithm to the scaled points
4. **Compute convex hull**: Apply Graham Scan for area comparison
5. **Calculate metrics**: Circle area (πr^2), hull area (shoelace formula), waste percentage
6. **Transform results**: Convert circle center back to geographic coordinates, radius to km
7. **Visualize**: Render points, circle, and hull overlaid

5.6 Area Comparison

The "wasted area" quantifies how much extra coverage the circular antenna provides beyond what is strictly necessary:

$$\text{Waste } \% = \frac{A_{\text{circle}} - A_{\text{hull}}}{A_{\text{circle}}} \times 100 \tag{5}$$

The convex hull represents the minimal convex region containing all parishes, while the circle must extend beyond to maintain its shape constraint.

5.7 Results

Metric	**Value**
Parishes analyzed	34
Optimal antenna latitude	−0.1046
Optimal antenna longitude	−78.6086
Coverage radius	39.76 km
Circle area	4,966.78 kmš
Convex hull area	2,765.79 kmš
Area waste	44.31%

Table 1: Antenna placement results for Quito parishes

The waste percentage (44.31%) reflects the geometric overhead of circular coverage: the enclosing circle must extend beyond the convex hull at all points where the hull boundary is not tangent to the circle. Rural parishes in the northwest (Pacto, Gualea, Nanegal) and southeast (Pintag, Amaguaña) define the circle's diameter, while the hull follows the irregular parish distribution more closely.

5.8 Discussion

The optimal antenna location (-0.1046, -78.6086) falls near the parish of Nono, northwest of urban Quito. This counterintuitive result arises because Welzl's algorithm minimizes the *maximum* distance to any parish, not the *average* distance. The remote northwestern parishes pull the center away from the population center.

For practical telecommunications deployment, this analysis suggests that:

- A single antenna requires approximately 40 km radius coverage
- About 44% of the circular coverage area extends beyond the parish convex hull
- Multiple smaller antennas would likely be more efficient

The convex hull provides a tighter bound (2,765.79 kmš), but real antenna coverage is inherently circular (or sector-based), making the enclosing circle the relevant constraint.

6 Conclusion

The three algorithms presented demonstrate different paradigms in computational geometry:

- **Graham Scan** exemplifies the power of sorting and incremental construction, achieving optimal complexity through careful ordering.
- **Bentley-Ottmann** showcases the sweep line technique, converting a global problem into a sequence of local updates with output-sensitive complexity.
- **Welzl** demonstrates randomized algorithms, where random ordering provides expected linear time despite worst-case exponential behavior.

The Quito antenna network application demonstrates that these theoretical algorithms have direct practical value. By combining Welzl's algorithm with real geographic data, we determined that covering all 34 Quito parishes requires an antenna with 39.76 km radius, with 44% area waste compared to the convex hull—highlighting the geometric overhead of circular coverage for irregularly distributed populations.

All implementations are available in the accompanying Zig codebase with SDL3 visualization for interactive exploration.

References

[1] R. L. Graham, *An efficient algorithm for determining the convex hull of a finite planar set*, Information Processing Letters, vol. 1, no. 4, pp. 132–133, 1972.

[2] J. L. Bentley and T. A. Ottmann, *Algorithms for reporting and counting geometric intersections*, IEEE Transactions on Computers, vol. C-28, no. 9, pp. 643–647, 1979.

[3] E. Welzl, *Smallest enclosing disks (balls and ellipsoids)*, New Results and New Trends in Computer Science, Lecture Notes in Computer Science, vol. 555, pp. 359–370, 1991.

[4] M. de Berg, O. Cheong, M. van Kreveld, and M. Overmars, *Computational Geometry: Algorithms and Applications*, 3rd ed., Springer-Verlag, 2008.

[5] Instituto Nacional de Estadística y Censos (INEC) Ecuador, *Parroquias del Ecuador - ArcGIS FeatureServer*, `https://services7.arcgis.com/iFGeGXTAJXnjq0YN/ArcGIS/rest/services/Parroquias_del_Ecuador/FeatureServer`, accessed March 2026.